

Public Key Cryptography

Dr. Yiwen Hu

Announcement

- Please double-check the information listed on the **Project Group** and **Paper Presentation Group** sheets
 - if your group info is incorrect or missing, please go ahead and correct it yourself.

https://docs.google.com/spreadsheets/d/1mIZobj2u5Y-U1ByppfC9GSLK_bwVjQhkghWIMz4LSgU/edit?usp=sharing



Announcement

Date	Lecture Topic	Slot	Paper Title	Venue	Group #
Nov. 2, 2026	Wireless Network Security (Wi-Fi)	1	Surveilling the Masses with Wi-Fi-Based Positioning Systems	IEEE S&P 2024	
		2	AirSnitch: Demystifying and Breaking Client Isolation in Wi-Fi Networks	NDSS 2026	
Nov. 4, 2026	Wireless Network Security (IoT, V2X)	1	From Virtual Touch to Tesla Command: Unlocking Unauthenticated Control Chains From Smart Glasses for	IEEE S&P 2024 (Distinguished Paper)	
		2	Hidden and Lost Control: on Security Design Risks in IoT User-Facing Matter Controller	NDSS 2025	
Nov. 9, 2026	Cellular Network Security (5G)	1	Uncovering Hidden Paths in 5G: Exploiting Protocol Tunneling and Network Boundary Bridging	ACM CCS 2025	
		2	Sni5Gect: A Practical Approach to Inject aNRchy into 5G NR	USENIX Security 2025	
Nov. 11, 2026	Cellular Network Security (NTN)	1	Direct-to-Cell Satellite Network without Satellite Navigation	ACM SIGCOMM 2025	
		2	Don't Look Up: There Are Sensitive Internal Links in the Clear on GEO Satellites	ACM CCS 2025 (Distinguished Paper)	
Nov. 16, 2026	Cellular Network Security (Emerging applications and services)	1	Insecurity of Lost/Stolen Phone Reporting Services: Vulnerabilities, Attacks, and Countermeasures	ACM MobiSys 2026 (Best Paper)	
		2	eSIMplicity or eSIMplification? Privacy and Security Risks in the eSIM Ecosystem	USENIX Security 2025	
Nov. 18, 2026	Machine Learning for Network Security	1	Hermes: Unlocking Security Analysis of Cellular Network Protocols by Synthesizing Finite State Machines	USENIX Security 2024	
		2	SoK: Decoding the Enigma of Encrypted Network Traffic Classifiers	IEEE S&P 2025	
Nov. 23, 2026	Adversarial Attacks on ML-Based Networked Systems	1	Practical Adversarial Attack on WiFi Sensing Through Unnoticeable Communication Packet Perturbation	ACM MobiCom 2024	
		2	Malicious Attacks against Multi-Sensor Fusion in Autonomous Driving	ACM MobiCom 2024	
Nov. 30, 2026	Security Risks of Agentic Network Operations	1	Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework	ACM SIGCOMM 2025	
		2	Investigating the Impact of Dark Patterns on LLM-Based Web Agents	IEEE S&P 2026	

Announcement

For each paper presentation group:

- Please take these two days to discuss within your group which papers on the paper list your group is interested in presenting.
- In class on this Wednesday (09/23), we will open a sign-up link where each group can register the paper they want to present.
 - Each paper can be presented by only one group, and sign-ups are first come, first served

Announcement

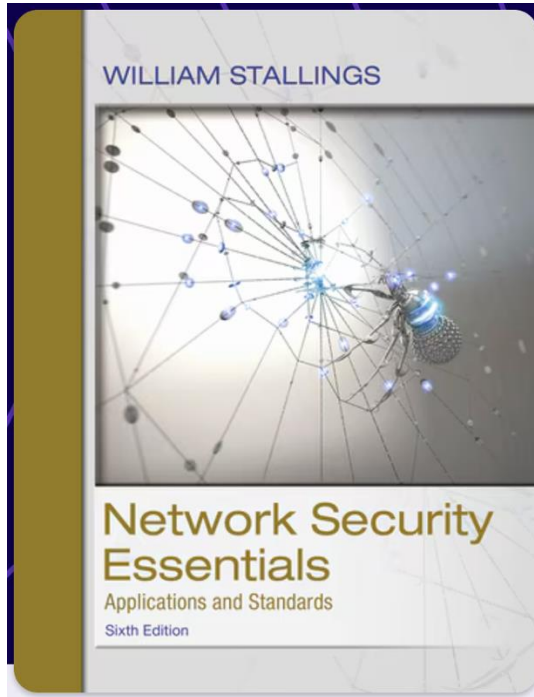
For paper presentation,

- You are allowed to use ChatGPT, Gemini, or similar tools to help you understand the technical content of the papers, **but you are not allowed to use them to directly generate your slides**
- Each group's presentation date follows the class date listed next to the paper
- Each presentation is **15 minutes** long: every group member must take part in presenting, though how you divide the time among members is up to you
- Presentations will be graded by the instructor, the TA, and the students in class. A grading rubric (metrics table) will be provided

Announcement

- Let these papers inspire your course group project brainstorm
 - No pressure. I am not expecting your projects to reach that level of quality
 - The goal is to give you a sense of what the network security research community has been working on most recently

Acknowledgement



You can find it on Blackboard

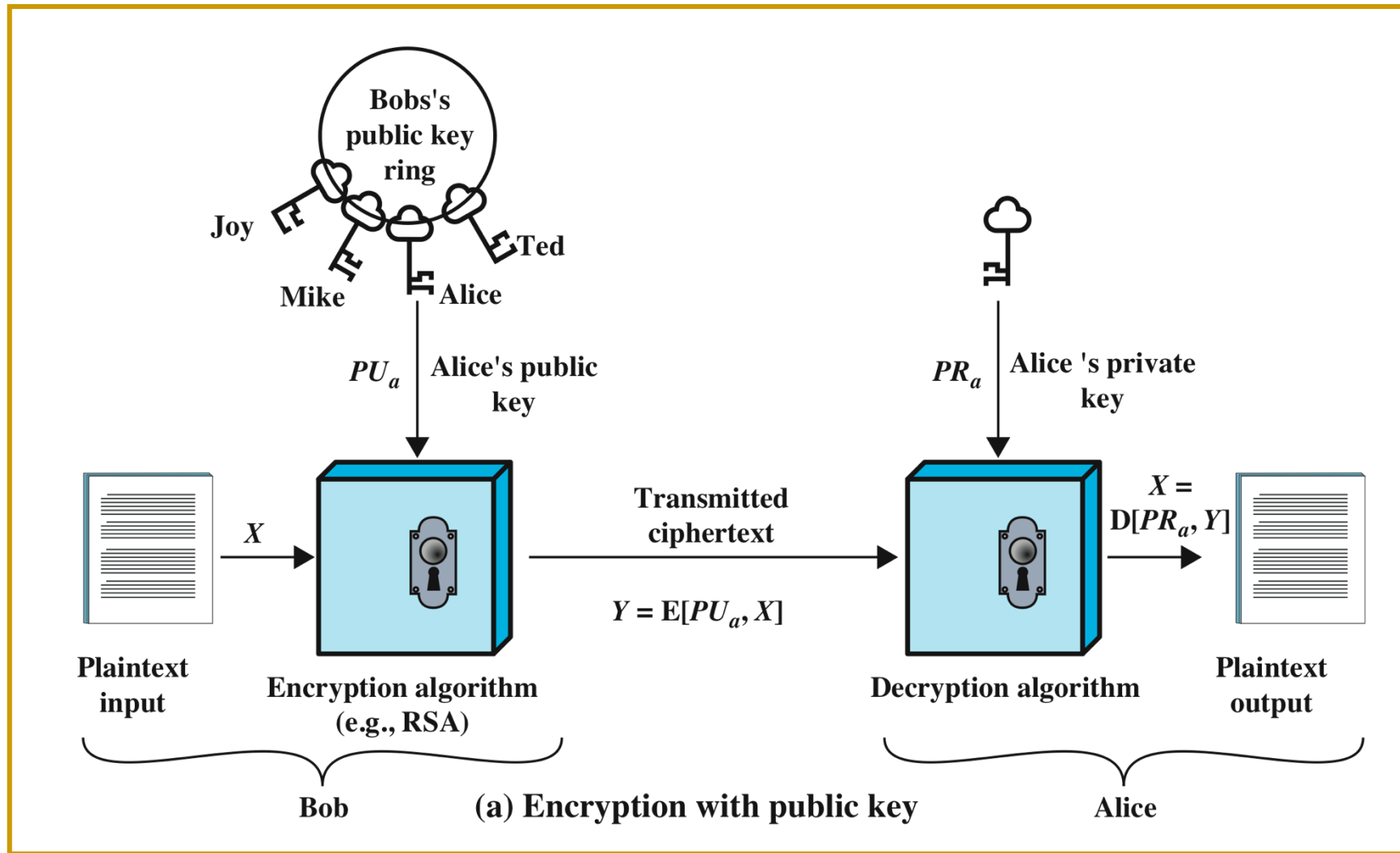
The content of slides is from the textbook and Dr. Hu's supplementary materials

Reminder: Please do not post course materials online or share them with others without the instructor's permission

Public-key cryptography applications

- Data confidentiality (encryption/decryption)
- Key management (key exchange / key distribution)
- Digital signature (integrity, authentication, non-repudiation)
- Entity authentication (e.g., TLS certificates, SSH login)

Data confidentiality



- The sender encrypts the data using the receiver's public key.
- Only the receiver, who possesses the corresponding private key, can decrypt the message.

Data integrity + authentication

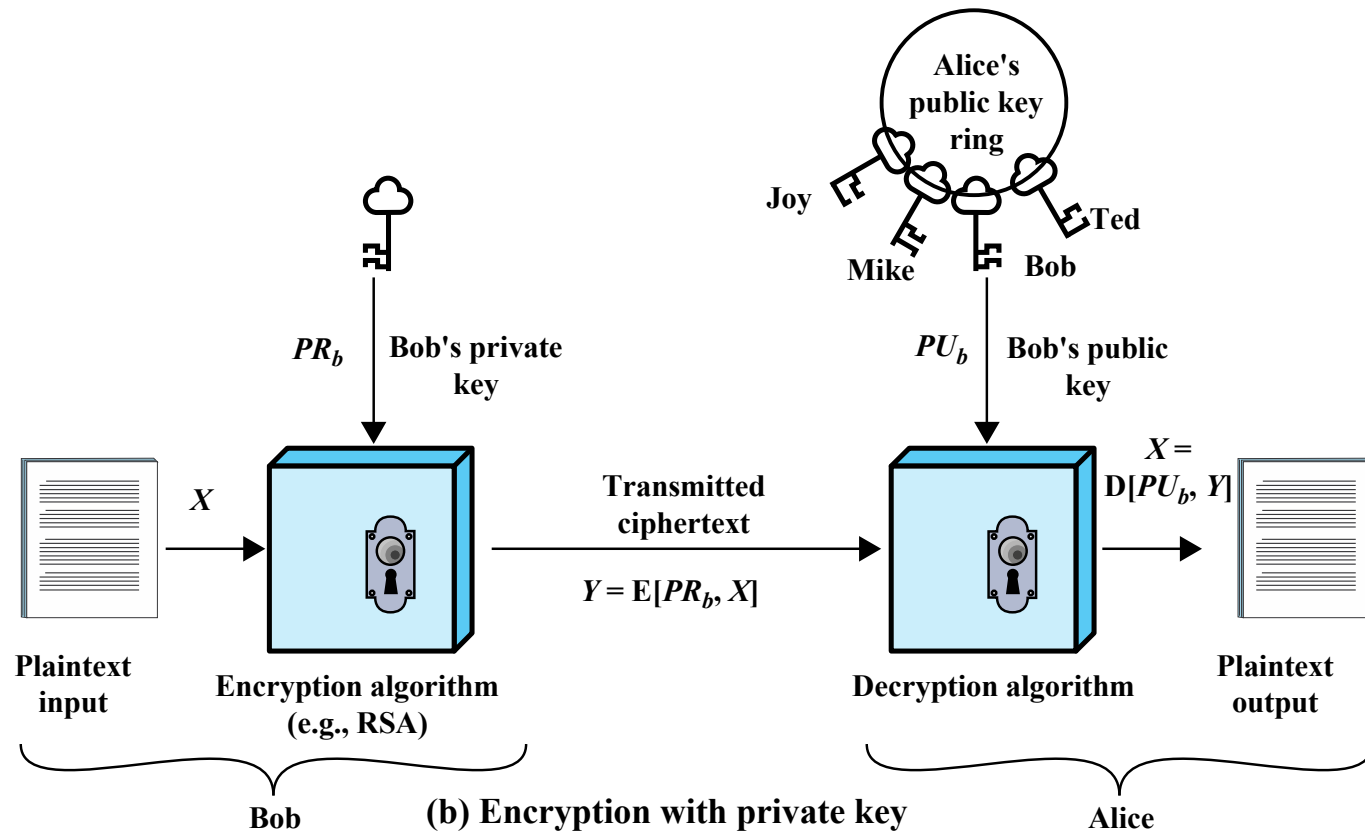
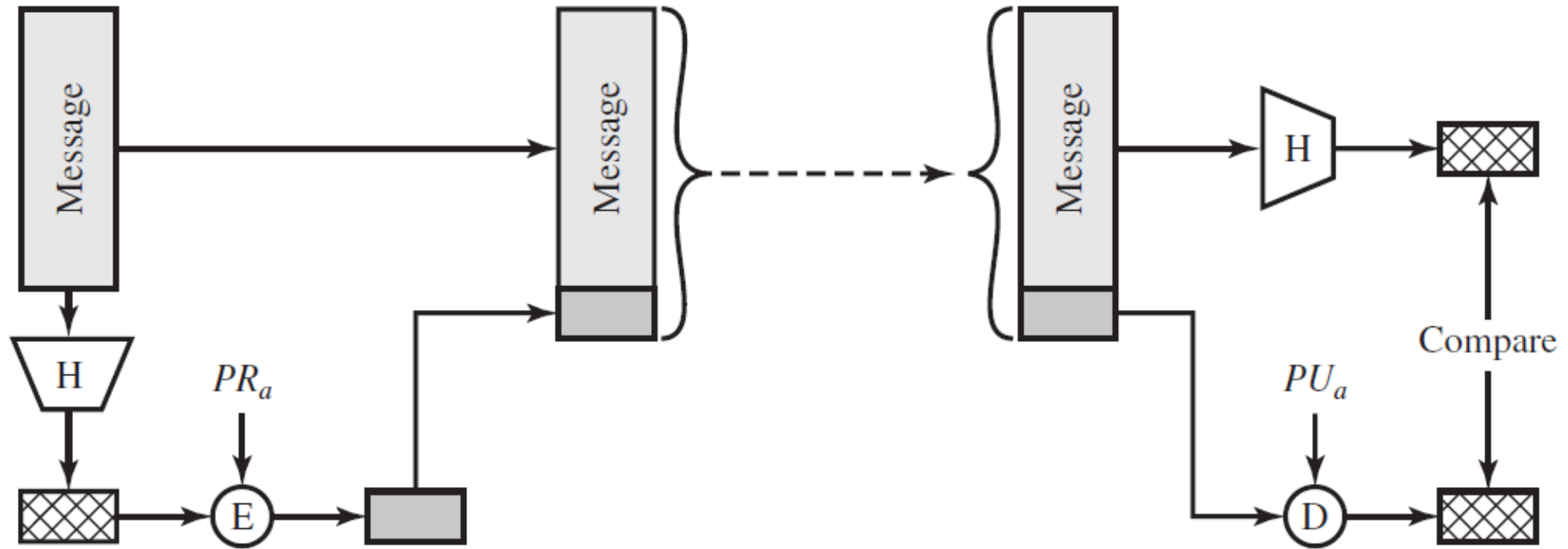


Figure 2.6 Public-Key Cryptography

- Sender encrypts data using his or her own private key
- Anyone who knows the corresponding public key will be able to decrypt the message

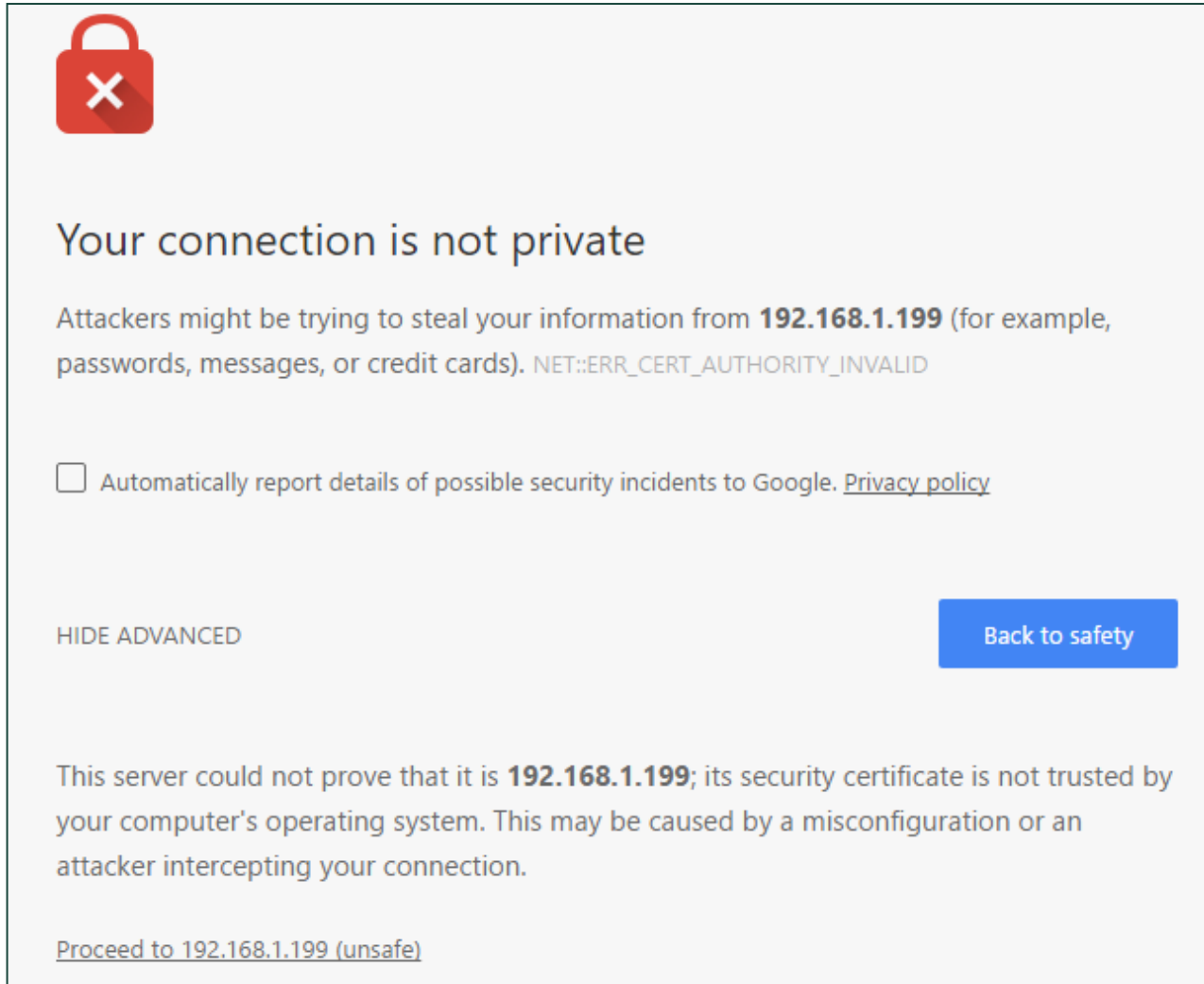
Digital Signatures



Digital Signature Standard (DSS):

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf>

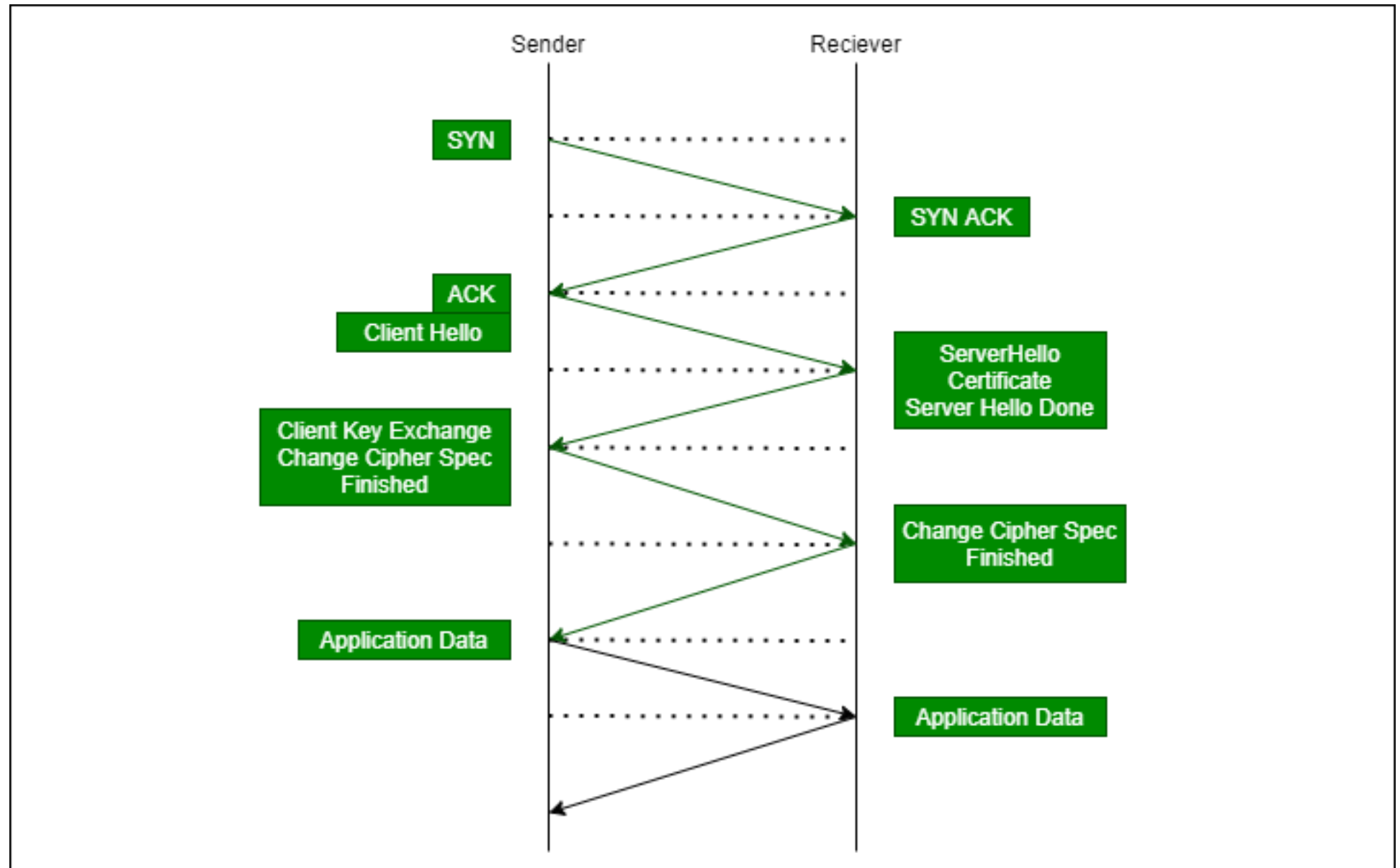
Key management



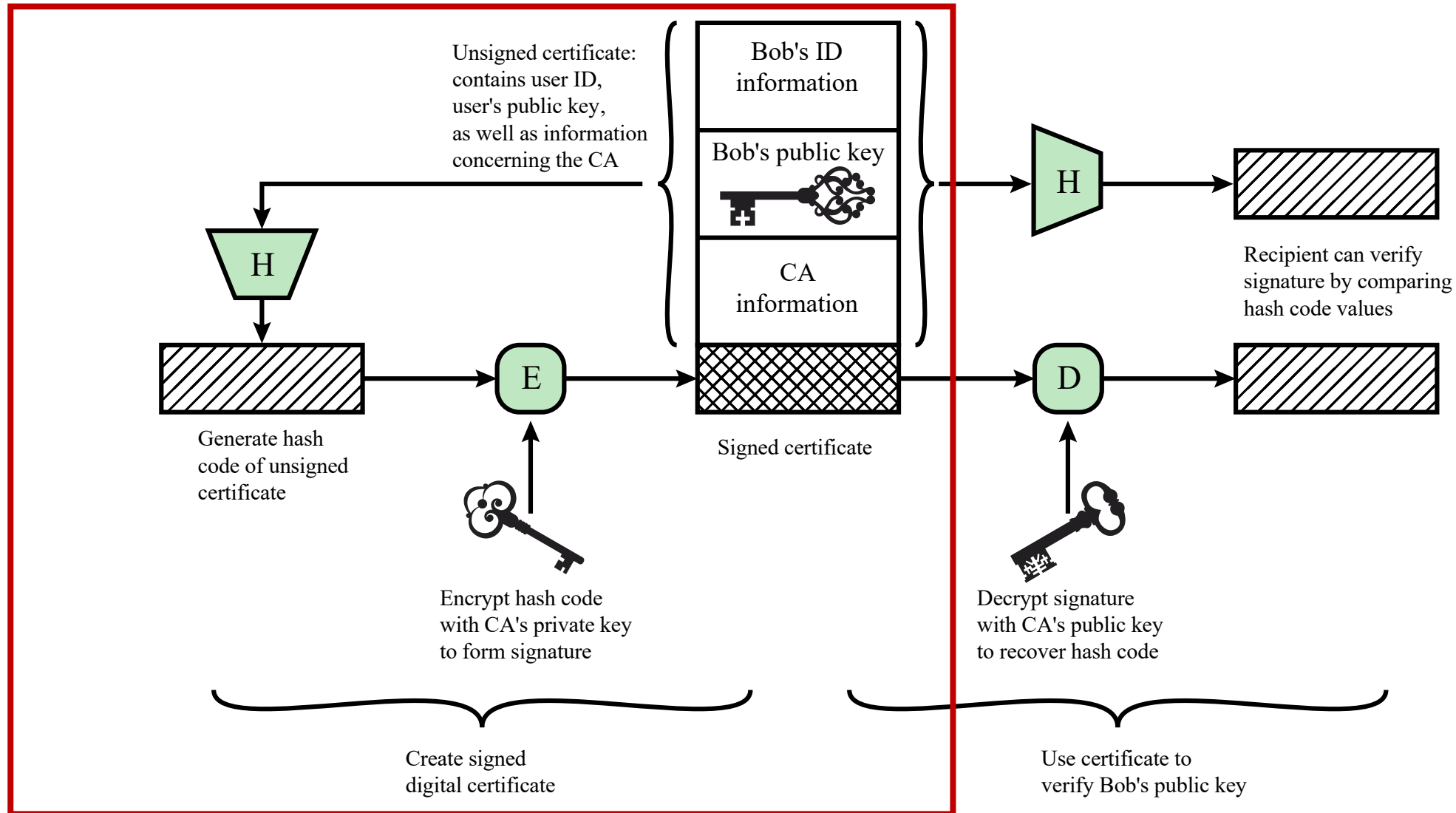
Public keys don't need to be secret,
but they must be authentic

Distributing a public key is easy;
proving who it belongs to is the hard
part

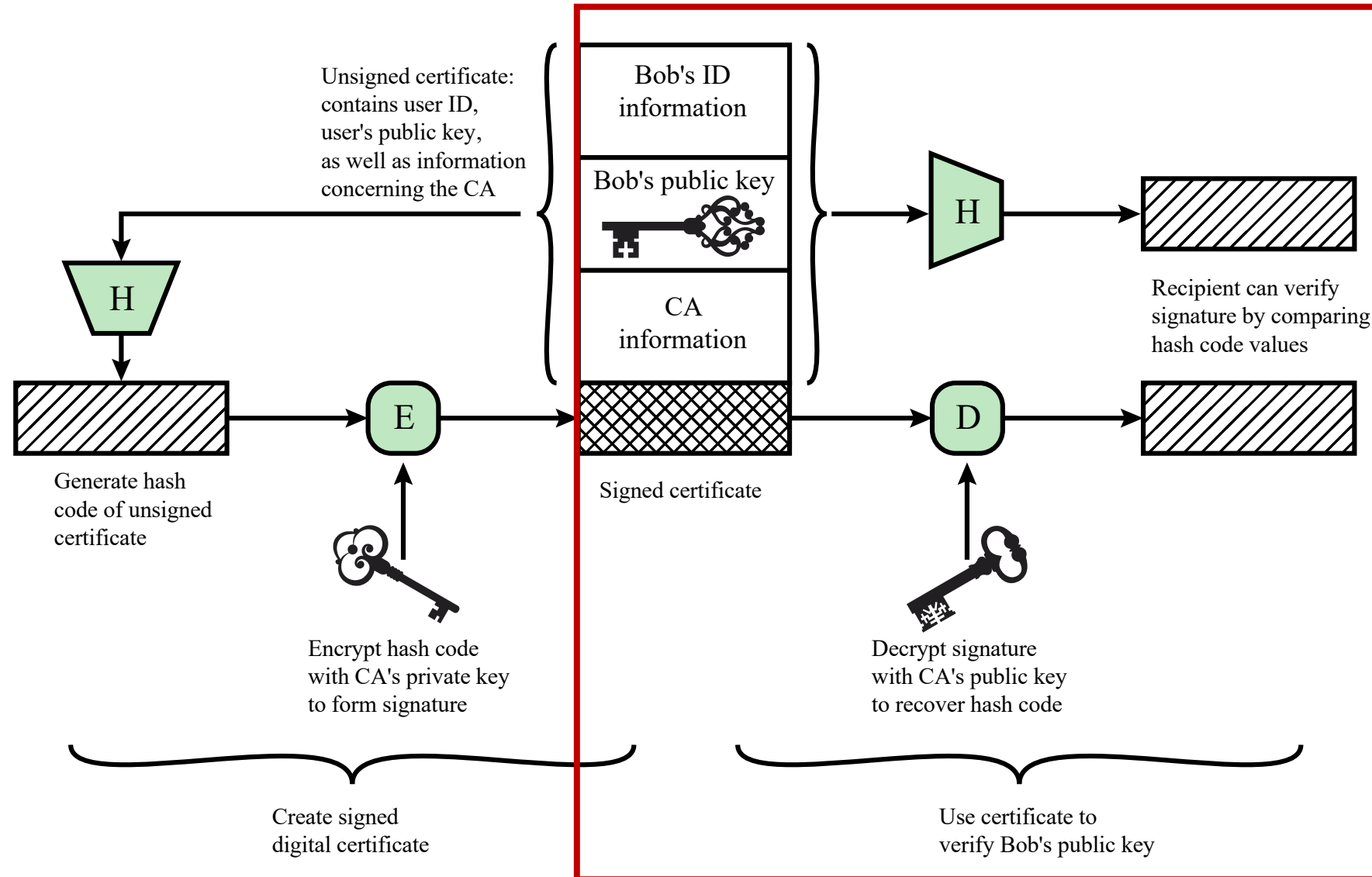
Transport Layer Security (TLS) Handshake



Key management



Key management



Key management

- **What is a CA?**
 - A CA is an entity that **binds a public key to an identity** by issuing a digitally signed **certificate** (X.509)
 - the CA verifies that the subject controls the claimed name (domain validation, organization validation, or extended validation) before signing
 - The signature makes the validation result verifiable by any relying party holding the CA's public key
- **Technically, anyone can be a CA**
 - One OpenSSL command → you have a root key and can sign certificates

Being a CA $\neq$ being a *trusted* CA

- Trusted = your public key is in the root store of the OS / browser
- Root stores are maintained by Mozilla, Apple, Microsoft, Google
- ~100+ root CAs are pre-installed and trusted by default

How to get into a root store?

- Meet CA/Browser Forum Baseline Requirements
- Pass independent audits every year (WebTrust / ETSI)
- ...

Examples

- Public CAs: DigiCert, Sectigo, GlobalSign, Let's Encrypt (free, non-profit)
- Tech companies: Google Trust Services, Apple, Microsoft, Amazon
- Private CAs: your company / university, trusted only on its own devices
- Self-signed = you are your own CA ($\rightarrow$ the browser warning on slide 10)

```
security find-certificate -a /System/Library/Keychains/SystemRootCertificates.keychain | grep '"labl"' |  
sed 's/.*="//;s/"$//' | sort
```

```
[yiwen@Mac ~ % security find-certificate -a /System/Library/Keychains/SystemRootCertif  
[| grep '"labl"' | sed 's/.*="//;s/"$//' | sort  
[    "labl"<blob>=0x4E65744C6F636B204172616E792028436C61737320476F6C64292046C59174616E  
[A16E79  "NetLock Arany (Class Gold) F\305\221tan\303\272s\303\255tv\303\241ny  
[AAA Certificate Services  
AC RAIZ FNMT-RCM  
ACCVRAIZ1  
Actalis Authentication Root CA  
AffirmTrust Commercial  
AffirmTrust Networking  
AffirmTrust Premium  
AffirmTrust Premium ECC  
Amazon Root CA 1  
Amazon Root CA 2  
Amazon Root CA 3  
Amazon Root CA 4  
Apple Root CA  
Apple Root CA - G2  
Apple Root CA - G3  
Apple Root Certificate Authority
```

This command lists the names of all root CA certificates pre-installed in macOS's system trust store

```
openssl s_client -connect www.umbc.edu:443 -servername www.umbc.edu </dev/null 2>/dev/null |  
openssl x509 -noout -text
```

```
yiwen@Mac ~ % openssl s_client -connect www.umbc.edu:443 -servername www.umbc.edu </dev/null 2>/dev/null | openssl x509  
-noout -text  
Certificate:  
  Data:  
    Version: 3 (0x2)  
    Serial Number:  
      06:75:cf:6e:42:e0:e2:97:0a:b5:5c:4e:a8:14:78:ff:42:ca  
    Signature Algorithm: sha256WithRSAEncryption  
    Issuer: C=US, O=Let's Encrypt, CN=YR2  
    Validity  
      Not Before: Aug 21 12:54:15 2026 GMT  
      Not After : Nov 19 12:54:14 2026 GMT  
    Subject: CN=about.umbc.edu  
    Subject Public Key Info:  
      Public Key Algorithm: rsaEncryption  
      RSA Public-Key: (2048 bit)  
      Modulus:  
        00:d4:c8:a2:1b:50:b4:c3:d2:df:35:ec:d8:65:f1:  
        67:9b:ac:02:ed:84:16:58:f4:b5:e0:d4:03:d3:0e:  
        20:83:07:94:0f:fd:f3:59:55:ff:cd:c7:68:41:52:  
        b9:7e:18:5f:b9:5d:4a:1f:4b:93:25:71:9e:5d:eb:  
        b8:df:2a:e8:f5:5c:a1:f6:c8:1a:68:a3:08:5d:ee:  
        0f:03:df:34:dc:06:78:01:d4:15:ee:d0:02:db:9d:  
        ce:f5:4f:4b:56:58:b8:fe:a2:76:f3:3c:b6:18:ee:  
        a0:13:ed:be:1d:8f:e9:ad:e7:3c:82:ca:97:3e:de:  
        95:af:ab:8c:3c:a7:61:52:42:3b:c6:3b:ed:2d:63:  
        d5:cd:50:54:39:4c:d1:2d:f3:64:ca:e1:fd:f7:f9:  
        0b:88:33:95:1e:35:78:d2:2a:20:5c:4c:59:6e:16:
```

This command performs a TLS handshake with www.umbc.edu, captures the certificate the server sends back, and prints its contents.

```
[yiwen@Mac ~ % openssl s_client -connect www.umbc.edu:443 -servername www.umbc.edu </dev/null 2>/dev/null | openssl x509
[ -noout -text
[Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      06:75:cf:6e:42:e0:e2:97:0a:b5:5c:4e:a8:14:78:ff:42:ca
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=US, O=Let's Encrypt, CN=YR2
    Validity
      Not Before: Aug 21 12:54:15 2026 GMT
      Not After : Nov 19 12:54:14 2026 GMT
    Subject: CN=about.umbc.edu
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public-Key: (2048 bit)
      Modulus:
        00:d4:c8:a2:1b:50:b4:c3:d2:df:35:ec:d8:65:f1:
        67:9b:ac:02:ed:84:16:58:f4:b5:e0:d4:03:d3:0e:
        20:83:07:94:0f:fd:f3:59:55:ff:cd:c7:68:41:52:
        b9:7e:18:5f:b9:5d:4a:1f:4b:93:25:71:9e:5d:eb:
        b8:df:2a:e8:f5:5c:a1:f6:c8:1a:68:a3:08:5d:ee:
        0f:03:df:34:dc:06:78:01:d4:15:ee:d0:02:db:9d:
        ce:f5:4f:4b:56:58:b8:fe:a2:76:f3:3c:b6:18:ee:
        a0:13:ed:be:1d:8f:e9:ad:e7:3c:82:ca:97:3e:de:
```

Overview

- RSA encryption/decryption algorithm
- Diffie-Hellman Key Exchange

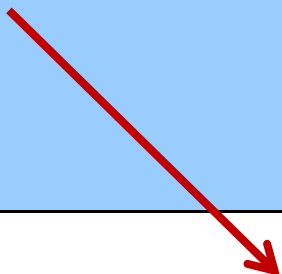
RSA Public-Key Encryption/Decryption

- By Rivest, Shamir & Adleman of MIT in 1977
- Best known and widely used public-key algorithm
- Theorems behind RSA
 - Modular arithmetic, Euler's totient function, Euler's product formula, Euler's theorem

The RSA Algorithm

Key Generation

Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \cdot q$	
Calculate $\phi(n) = (p-1)(q-1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$de \bmod \phi(n) = 1$
Public key	$KU = \{e, n\}$
Private key	$KR = \{d, n\}$



Encryption

Plaintext:	$M < n$
Ciphertext:	$C = M^e \pmod{n}$

Decryption

Ciphertext:	C
Plaintext:	$M = C^d \pmod{n}$

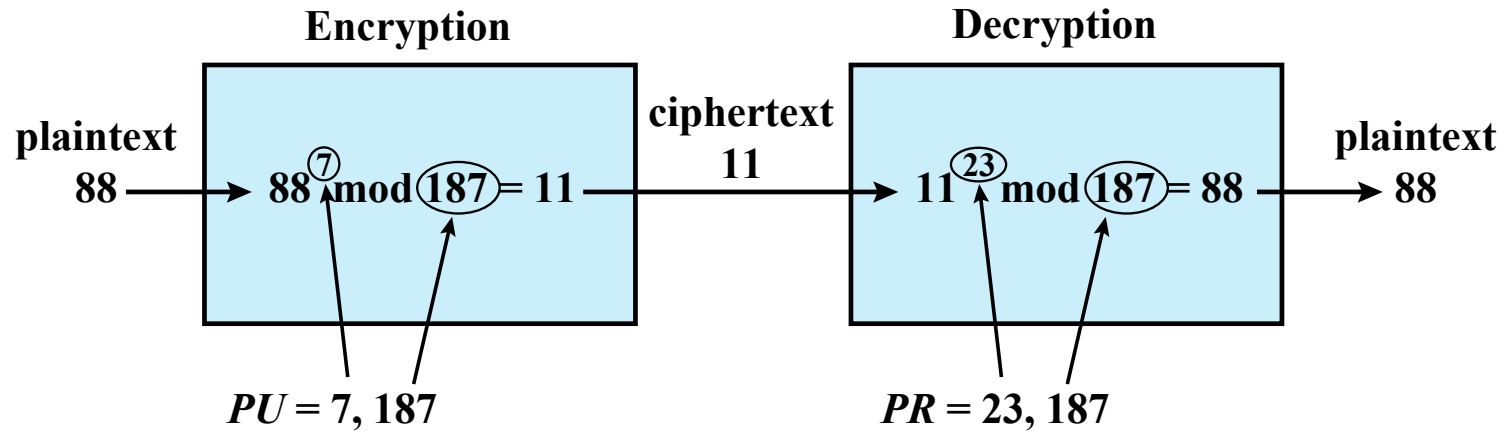
$$(d * e) \bmod \phi(n) = 1$$

Greatest Common Divisor (GCD)

it implies $d * e = k * \phi(n) + 1$, where k is an integer

$$d = (k * \phi(n) + 1) / e$$

Example of RSA Algorithm



Public Key is the pair (e, n)
Private Key is the pair (d, n)

RSA Factoring Challenge

Encryption	
Plaintext:	$M < n$
Ciphertext:	$C = M^e \pmod n$

Decryption	
Ciphertext:	C
Plaintext:	$M = C^d \pmod n$

Key Generation	
Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \cdot q$	
Calculate $\phi(n) = (p-1)(q-1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$de \pmod{\phi(n)} = 1$
Public key	$KU = \{e, n\}$
Private key	$KR = \{d, n\}$

- For an adversary, he/she can get public key $\{e \text{ and } n\}$ and C (ciphertext).
- To decipher C , the adversary needs to infer private key d so that he/she can infer M (plaintext) = $C^d \pmod n$
- How to discover d (private key)?
 - $d = (k \cdot \phi(n) + 1) / e$, where k is an integer
 - There is no efficient solution to calculate $\phi(n)$. The best algorithm requires $O(\sqrt{n})$ if you cannot factorize n first (recall: $\phi(n) = (p-1) \cdot (q-1)$).

Question: How to encrypt data by RSA?

E.g., encrypting “abc”

Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex
(sp)	32	0040	0x20	@	64	0100	0x40	`	96	0140	0x60
!	33	0041	0x21	A	65	0101	0x41	a	97	0141	0x61
"	34	0042	0x22	B	66	0102	0x42	b	98	0142	0x62
#	35	0043	0x23	C	67	0103	0x43	c	99	0143	0x63
\$	36	0044	0x24	D	68	0104	0x44	d	100	0144	0x64
%	37	0045	0x25	E	69	0105	0x45	e	101	0145	0x65
&	38	0046	0x26	F	70	0106	0x46	f	102	0146	0x66
'	39	0047	0x27	G	71	0107	0x47	g	103	0147	0x67
(	40	0050	0x28	H	72	0110	0x48	h	104	0150	0x68
)	41	0051	0x29	I	73	0111	0x49	i	105	0151	0x69
*	42	0052	0x2a	J	74	0112	0x4a	j	106	0152	0x6a
+	43	0053	0x2b	K	75	0113	0x4b	k	107	0153	0x6b
,	44	0054	0x2c	L	76	0114	0x4c	l	108	0154	0x6c
-	45	0055	0x2d	M	77	0115	0x4d	m	109	0155	0x6d
.	46	0056	0x2e	N	78	0116	0x4e	n	110	0156	0x6e
/	47	0057	0x2f	O	79	0117	0x4f	o	111	0157	0x6f
0	48	0060	0x30	P	80	0120	0x50	p	112	0160	0x70
1	49	0061	0x31	Q	81	0121	0x51	q	113	0161	0x71
2	50	0062	0x32	R	82	0122	0x52	r	114	0162	0x72
3	51	0063	0x33	S	83	0123	0x53	s	115	0163	0x73
4	52	0064	0x34	T	84	0124	0x54	t	116	0164	0x74
5	53	0065	0x35	U	85	0125	0x55	u	117	0165	0x75
6	54	0066	0x36	V	86	0126	0x56	v	118	0166	0x76
7	55	0067	0x37	W	87	0127	0x57	w	119	0167	0x77
8	56	0070	0x38	X	88	0130	0x58	x	120	0170	0x78
9	57	0071	0x39	Y	89	0131	0x59	y	121	0171	0x79
:	58	0072	0x3a	Z	90	0132	0x5a	z	122	0172	0x7a
;	59	0073	0x3b	[	91	0133	0x5b	{	123	0173	0x7b
<	60	0074	0x3c	\	92	0134	0x5c		124	0174	0x7c
=	61	0075	0x3d	]	93	0135	0x5d	}	125	0175	0x7d
>	62	0076	0x3e	^	94	0136	0x5e	~	126	0176	0x7e
?	63	0077	0x3f	_	95	0137	0x5f				

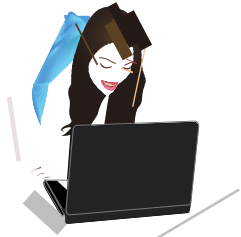
Overview

- RSA encryption/decryption algorithm
- Diffie-Hellman Key Exchange

Diffie-Hellman Key Exchange

- Practical method to exchange a secret key securely that can then be used for subsequent encryption of messages
- By Diffie and Hellman in 1976 along with the exposition of public key concepts
- Security relies on difficulty of computing discrete logarithms

Diffie-Hellman Key Exchange



Alice



Bob

Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Alice generates a private key X_A such that $X_A < q$

Bob generates a private key X_B such that $X_B < q$

Alice calculates a public key $Y_A = \alpha^{X_A} \bmod q$

Bob calculates a public key $Y_B = \alpha^{X_B} \bmod q$

Alice receives Bob's public key Y_B in plaintext

Bob receives Alice's public key Y_A in plaintext

Alice calculates shared secret key $K = (Y_B)^{X_A} \bmod q$

Bob calculates shared secret key $K = (Y_A)^{X_B} \bmod q$

